THE ART OF READABLE CODE · CHAPTER 10

Extracting Unrelated Subproblems

Dustin Boswell & Trevor Foucher

Engineering guide · Writing modular, self-contained functions



PRESENTATION TEAM

Meet the Team

Chapter 10 · Extracting Unrelated Subproblems

MS

MONY SOTHYRA

ID: e20220229

NP

NOV Panhavath

ID: e20221643

PV

PRIM VEASNA

ID: e20221402

DB

DOEUN Bunheng

ID: e20221528

CORE PHILOSOPHY

One function should have one main job

When sub-tasks clutter the primary workflow, decompose and delegate.

BEFORE EXTRACTION

One Big Tangled Function

Main logic, trigonometric calculations, payload formatting, and type conversions mixed into a single long scope.

Hard to skim · Fragile tests · Zero reuse

AFTER EXTRACTION

Focused Orchestrator + Helpers

A clean high-level function that tells a clear story, delegating isolated mathematical and data chores to pure utilities.

Instant readability · Unit testable · Portable logic

EXTRACTION HEURISTIC

Find the work that does not belong

A straightforward 3-step test to identify extraction candidates.

01

1. State the main goal

Ask what this block of code is trying to achieve at a high architectural level.

02

2. Inspect every line

Ask whether each statement directly serves that main goal or merely solves an incidental puzzle.

03

3. Extract the subproblem

If enough lines solve an unrelated subproblem, carve them into an independent helper function.

CASE STUDY

findClosestLocation() does two jobs

Main Goal: Iterate through locations and pick the one closest to a target coordinate.

The Intrusion: Spherical trigonometry clutters the search loop and obscures intent.

The geometry is completely agnostic to location objects and business context.

// TANGLED: SPHERICAL MATH CLUTTERS SELECTION

```
function findClosestLocation(target, locations){
 let closest = null, minDistance = Infinity;

 for (const loc of locations){
  //Unrelated subproblem: Spherical distance math
  const lat1 = target.lat * (Math.PI/180);
  const lat2 = loc.lat * (Math.PI/180);
  const dLon = (loc.lng - target.lng) * (Math.PI/180);
  const dist = Math.acos(
   Math.sin(lat1) * Math.sin(lat2) +
   Math.cos(lat1) * Math.cos(lat2) * Math.cos(dLon)
  ) * 6371;

  if (dist < minDistance){
   minDistance = dist;
   closest = loc;
  }
 }
 return closest;
}
```

REFACTORED RESULT

After extraction, the code reads like a story

// PRIMARY: CLEAR SELECTION STORY

```
function findClosestLocation(target, locations){
 let closest = null;
 let minDistance = Infinity;

 for (const loc of locations){
  const dist = sphericalDistance(target, loc);
  if (dist < minDistance){
   minDistance = dist;
   closest = loc;
  }
 }
 return closest;
}
```

// EXTRACTED HELPER: PURE & TESTABLE

```
function sphericalDistance(p1, p2){
 const toRad = deg => deg * (Math.PI/180);
 const lat1 = toRad(p1.lat), lat2 = toRad(p2.lat);
 const dLon = toRad(p2.lng - p1.lng);

 return Math.acos(
  Math.sin(lat1)*Math.sin(lat2)+
  Math.cos(lat1)*Math.cos(lat2)*Math.cos(dLon)
 )*6371;
}
```


Readable: The reader skims the loop instantly · Testable: Geometry can be unit-tested without mock locations · Reusable: Works for any lat/lng points

REUSABLE UTILITIES

Turn repeated low-level work into a utility

Do not let low-level buffer allocations, file descriptors, and stream handles distract from business operations.

The Rule of Thumb

If you often wish the standard library had a specific helper, write it once and share it across the codebase.

Self-Descriptive Naming

A wrapper named `ReadFileToString()` states the outcome directly, hiding boilerplate error checks.



SEPARATION OF CONCERNS

General-purpose helpers make intent visible

Isolate presentation, normalization, and wire protocols from workflow coordination.



Ajax Request

Dispatch Ajax query.



Receive Response

Get unformatted JSON.



format_pretty(obj)

Extracted subproblem.



Handle Response

Update the UI view.

Understand · Test · Improve · Reuse · Handle edge cases

Wrap formatting complexity so the calling code can stay about the response.

ENCAPSULATION

Hide ugly interfaces behind clean wrappers

Browser cookies are a good example of complicated implementation details.

```
get_cookie(name)
```

Read

Encapsulates semicolon splitting, key decoding, and trimming from `document.cookie`.

```
set_cookie(name, value, days)
```

Write

Encapsulates max-age math, date conversions, URL encoding, and path defaults cleanly.

```
delete_cookie(name)
```

Remove

Abstracts the non-intuitive convention of setting the expiration timestamp into the past.

Expose a small interface; keep `document.cookie` handling inside.

OVER-ENGINEERING WARNING

Do not extract until the code gets clearer

Extraction is not free. Creating too many one-line wrappers forces readers to hop through files to follow basic flow.

Extraction should reduce mental load, not multiply call stacks.

THE PITFALL: CALL TUNNELING

```
url_safe_encrypt_obj(obj)
└── url_safe_encrypt_str(str)
    └── encrypt(data)
        └── make_cipher()
```

Know the Boundary

- Useful Extraction: Separates a truly independent domain (math, serialization, parsing).
- Fragmentation: Arbitrarily slicing a simple 10-line sequence into five micro-functions.

SUMMARY & TAKEAWAY

Keep the main code focused

01

Identify main goal

02

Find unrelated work

03

Extract helper functions

04

Keep main logic simple

Readable

Testable

Reusable

Maintainable

Keep the main code focused on what the program is trying to do, and move unrelated details into well-named helper functions.